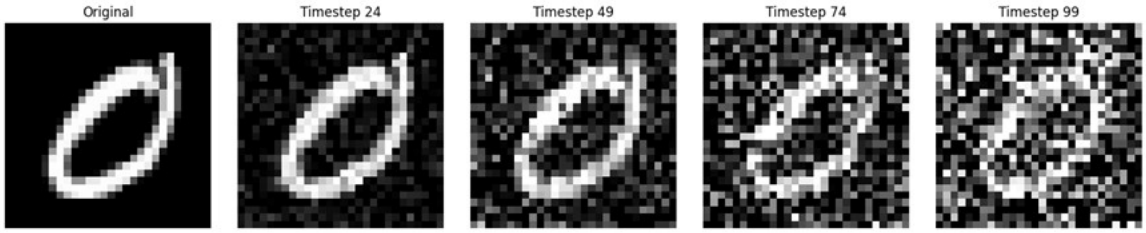
Creating Images with Diffusion Models — Assessment Report by Esther Odaibo

KING OWUSU - ASSESSMENT QUESTIONS

1. Understanding Diffusion

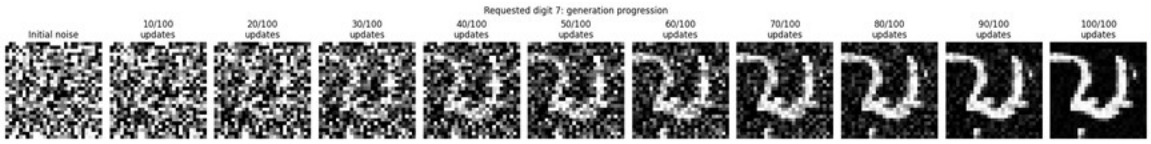
Forward diffusion. The forward process slowly destroys an image by mixing in Gaussian noise: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \varepsilon$. As t grows, the share of the original image shrinks and the share of noise grows. In the figure below, the "0" is still clearly a "0" at timestep 24, has a grainy outline at 49, is mostly broken up at 74, and is only a faint ghost at 99.



With β running from 0.0001 to 0.02 over only 100 steps, $\bar{\alpha}$ at the last step is about 0.36, so the image keeps roughly 60% of its signal ($\sqrt{\bar{\alpha}} \approx 0.60$). That is why a faint outline survives at step 99: the schedule does not quite reach pure noise.

Why add noise gradually? Each small step only needs to be undone a little, which is much easier than turning static into a digit in one jump. The model can learn one simple rule: predict the noise that was just added. Training picks a random timestep for every image, so the model sees every noise level. During generation, many small steps let the model fix its own mistakes and add detail progressively, coarse layout first and fine strokes later. One giant step would force the network to guess the whole image at once, which usually gives blurry averages.

When is the image first recognizable? In the step-by-step figure for digit 7, the image is static through about 20–30% of the process, a bold curved stroke appears at about 40–50%, and the overall shape is clear by about 70%. The last 30% mostly sharpens edges and removes grain. Digits 0 and 3 showed the same pattern, with structure appearing at about 50% and 50–60%. It varies by image: the 7 committed to its shape earliest, while the 3 stayed ambiguous longer. Here "recognizable" means a coherent stroke shape; the final shapes are not clean, readable digits (see Section 5).



2. Model Architecture

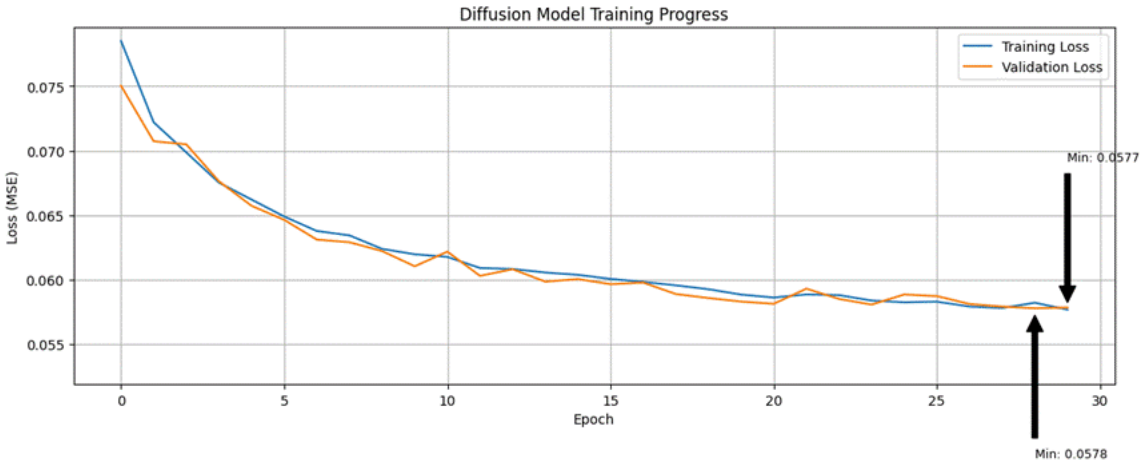
Why a U-Net? The model must output an image the same size as its input (the predicted noise), and it needs both global context (what digit is this?) and pixel-level detail (where exactly is the noise?). The down path shrinks 28×28 → 14×14 → 7×7 so each filter sees more of the image, and the up path expands back to 28×28. Compared with a plain stack of same-resolution convolutions, this gives a large receptive field cheaply. Compared with an encoder-only network, it keeps spatial layout and produces an output of the right shape.

Skip connections. A skip connection saves a feature map from the down path and concatenates it onto the matching up-path feature map, which is why `UpBlock`'s transposed convolution takes `2 * in_chs` channels. Downsampling throws away fine spatial detail; skips hand it straight to the decoder so it does not have to be rebuilt from the compressed 7×7 map. They also give gradients a short path back through the network, which helps training. In our model the skips are the 64-channel 14×14 map and the 128-channel 7×7 map, saved after each `DownBlock`. One limitation: there is no skip at full 28×28 resolution (the `initial_conv` output is not saved), which likely costs fine detail.

Class conditioning. The digit label becomes a one-hot vector (3 → [0,0,0,1,0,0,0,0,0,0]). `EmbedBlock` passes it through two linear layers with a GELU and reshapes it to [B, 128, 1, 1]. In `UNet.forward` it is multiplied by `c_mask` (1 = use the label, 0 = drop it) and added to the bottleneck feature map, so it broadcasts over every spatial position. The time step is handled the same way (sinusoidal embedding, linear layer, GELU, added at the same place). The decoder is therefore influenced by "which digit" and "which step". The mask would enable classifier-free guidance, but training here always uses mask = 1.

3. Training Analysis

What the loss tells us. The loss is the mean-squared error between the true noise and the predicted noise; lower means the model is better at finding the noise in a noisy image. Training loss fell from 0.0785 to 0.0577 (26.5% lower) and validation loss from 0.0750 to 0.0578. The two curves track each other closely, so there is no sign of overfitting. The curve flattens after about epoch 20 at roughly 0.058–0.059, so more epochs at this learning rate would give diminishing returns.



A caution: the loss is averaged over all timesteps, and a small loss does not guarantee good-looking samples. Our samples show this.

Image quality during training. Loss improved steadily, with the biggest drop in the first five epochs. By the end the model produces crisp white strokes on a clean black background with the right overall MNIST style (stroke width, centered shape), but the strokes do not form reliable digits. Correct digit identity is what did not appear.

Why the time embedding? The same network must denoise at every noise level. At $t \approx 95$ the right action is to pull a rough shape out of almost nothing, and at $t \approx 5$ it is to clean up a few grains of noise. Without a time signal the network cannot tell which situation it is in, because moderately noisy images look similar at different steps. The sinusoidal embedding gives each step a unique, smooth vector (nearby steps get similar vectors), and the network learns what to do with it, so it knows where it is in the denoising process and how strongly to correct.

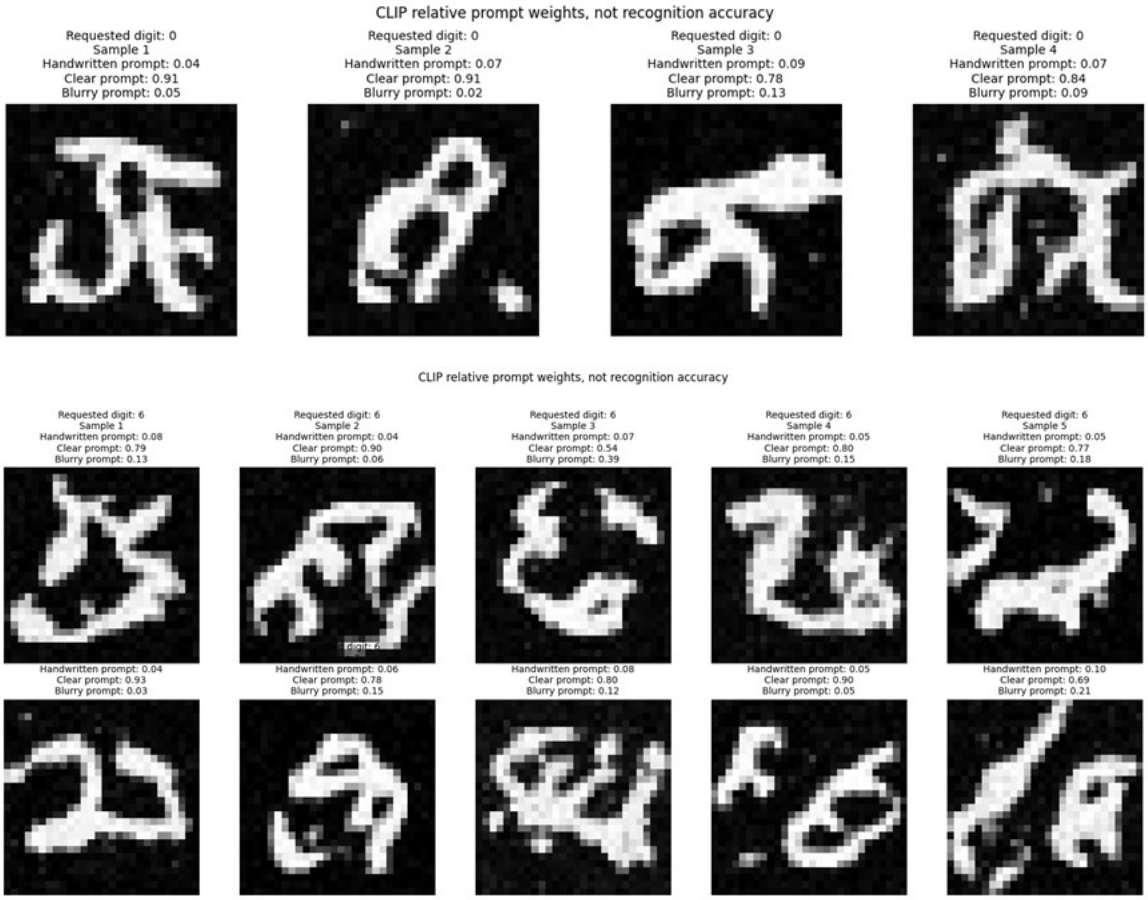
4. CLIP Evaluation

CLIP scored every generated image as "clear" (mean weight 0.789 across 10 digit-6 samples), but that measures how clean the picture looks, not whether it is the requested digit. Each image is compared with three prompts (handwritten, clear, blurry), and the weights sum to about 1; the notebook's own figure titles say "relative prompt weights, not recognition accuracy".

Requested digit	Samples	Mean "clear" weight	Range of "clear" weight
0	4	0.86	0.78 to 0.91
1	4	0.89	0.86 to 0.92
6	10	0.79	0.54 to 0.93

The highest scores were 0.93 (digit 6, sample 6), 0.92 (digit 1, samples 1 and 3) and 0.91 (digit 0, samples 1 and 2). The lowest was digit 6, sample 3, with clear 0.54 and blurry 0.39; it has thin, fragmented, disconnected strokes. High scores on images that are not readable as the

requested digit show that the metric rewards clean, high-contrast rendering rather than correctness.



Hypothesis. Images with thick, connected, high-contrast strokes and little speckle score higher on "clear", while broken, thin or scattered fragments score lower and drift toward "blurry". For digits, shapes with one dominant stroke (like 1) should be easier, and shapes with closed loops or several parts (6, 8, 9, 0) harder, because the model must get several parts right in the right relation. This is untested beyond the digits shown; a test would score 50 or more samples per digit and compare the averages.

Using CLIP to improve generation. Proposed technique: CLIP re-ranking (best-of-N) with digit-specific prompts. For each requested digit k, generate N = 16 candidates and score each against ten prompts ("a handwritten digit 0" through "a handwritten digit 9"). Keep only candidates where prompt k has the highest probability (zero-shot classification), then return the top few by that probability. This measures correctness directly, which the clear/blurry prompts do not. A stronger version would use CLIP's gradient to nudge x_t toward the target prompt during denoising, and classifier-free guidance using the existing c_mask is another cheap option.

5. Practical Applications

Real-world uses. Diffusion models are used for image generation and editing, data augmentation (for example synthetic medical or handwriting images when real data is scarce), denoising and restoration (old photos, MRI and CT noise reduction), inpainting and super-

resolution, and, with modified architectures, audio, video and molecule design. Class-conditional generation like ours is the simplest version of "generate X on request".

Limitations of our model.

- Samples do not reliably look like the requested digit; most of the 40-image grid is not readable as digits.
- Conditioning is injected only once, at the 7×7 bottleneck, and the time embedding is only 8-dimensional, so guidance about both digit and noise level is weak.
- There is no full-resolution skip connection, which limits fine detail.
- Only 100 steps, and the schedule ends at $\bar{\alpha} \approx 0.36$ instead of near 0, so generation starts from pure noise that training never saw.
- The model is small (1.5 million parameters), trained for 30 epochs, and the loss had plateaued.
- The CLIP evaluation measures clarity, not correctness, so it overstates quality, and there is no real accuracy measurement.

Three improvements, in priority order.

1. Inject time and class conditioning at every resolution level (inside each Down and Up block) and use a larger time embedding, for example 128 dimensions. This most directly addresses weak digit control and weak step awareness.
2. Fix the schedule and add a full-resolution skip: use more steps (300 to 1,000) or a cosine schedule so $\bar{\alpha}$ at the last step is near 0, and save the initial_conv output as a 28×28 skip. This removes the train-versus-generate mismatch and restores fine detail.
3. Add classifier-free guidance and a real evaluation: randomly drop the label during training (using the existing mask), sample with a guidance scale, and measure correctness with a small MNIST classifier and digit-specific CLIP prompts, so improvements show up as accuracy rather than only clarity.

Additional observations and generated examples

The grid below shows four samples for each requested digit 0 to 9 from the trained model. Stroke thickness and a clean black background match MNIST, but few samples read as the requested digit, which is the main finding to fix in any follow-up.



- **Training was stable.** Training and validation loss moved together for all 30 epochs, and the best validation loss was 0.0578.
- **Loss alone is a weak quality signal.** Validation loss was already flat at about 0.058 while samples were still not readable digits.
- **CLIP is better at judging clarity than correctness.** In every panel we inspected, the "clear" prompt won, including for images that do not resemble the requested digit.
- **The noise schedule is a likely cause.** The forward process ends at about 60% signal rather than pure noise, so the sampler starts from a distribution the model was never trained to denoise.
- **Experiments not run.** The bonus challenges (architecture changes, CLIP-guided selection, style conditioning) were not attempted in this report.

Observations and Analysis (King Owusu)

What happened. The model learned its training task well. Training loss fell from 0.0785 to 0.0577 and validation loss from 0.0750 to 0.0578 over 30 epochs. The two curves tracked each other, so there was no overfitting. The curve flattened after about epoch 20.

What did not happen. Low loss did not produce readable digits. In the 40-image grid (four samples for each digit 0 to 9), the strokes are clean and MNIST-like, but most samples do not look like the requested digit.

Why this probably happened

- Time and class information are added only once, at the 7×7 bottleneck, so the model gets weak guidance about which digit to draw.
- The time embedding is only 8-dimensional.
- There is no skip connection from the full-size 28×28 layer, which limits fine detail.
- The noise schedule ends with about 60% of the image signal left ($\bar{\alpha} \approx 0.36$). Generation starts from pure noise that training never saw.

CLIP results. The "clear" prompt won for all 10 digit-6 samples (mean weight 0.789). The lowest score was digit 6, sample 3 (clear 0.54, blurry 0.39), which has thin, broken strokes. CLIP judges how clean an image looks, not whether it is the right digit, so these scores overstate quality.

Main takeaway. Loss and CLIP clarity are both weak quality signals here. A real check, such as a small MNIST classifier that predicts the digit in each generated image, would measure correctness directly.

Next steps. (1) Add conditioning in every block, (2) fix the noise schedule and add a full-size skip connection, (3) add classifier-free guidance and an accuracy-based evaluation.

Generated Image Examples and Evaluations

| Group 4 | ITAI 2376

We evaluated the generated images for visual clarity, resemblance to the requested digit, and variation between samples. The examples show visible handwriting-like strokes, but the shapes do not consistently match the requested numbers.

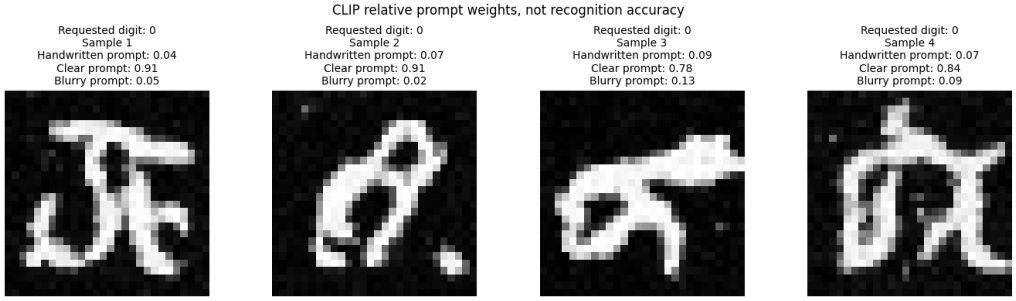


Figure 1. Four generated samples requested as digit 0.

Some samples contain loops, but additional lines and disconnected marks make them difficult to recognize as zeros. Their CLIP clear-prompt weights are 0.91, 0.91, 0.78, and 0.84, respectively. These high weights do not establish that the images contain the correct digit.

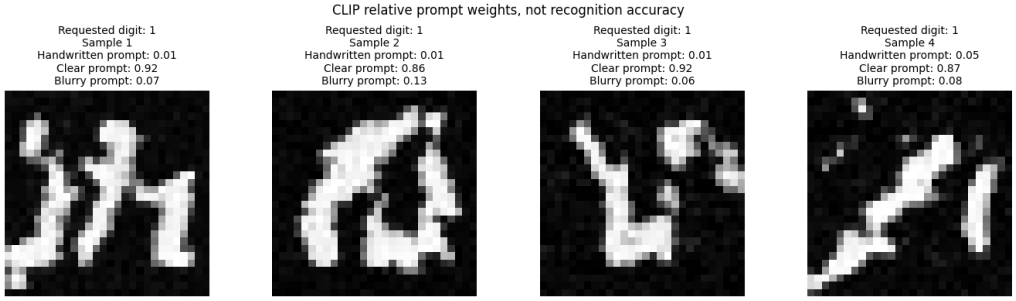


Figure 2. Four generated samples requested as digit 1.

These samples have visible strokes, but several contain branches or multiple connected parts that do not resemble a typical handwritten 1. Their clear-prompt weights are 0.92, 0.86, 0.92, and 0.87, respectively. Visual inspection indicates that the model did not reliably follow the requested class label.

Generation Progression

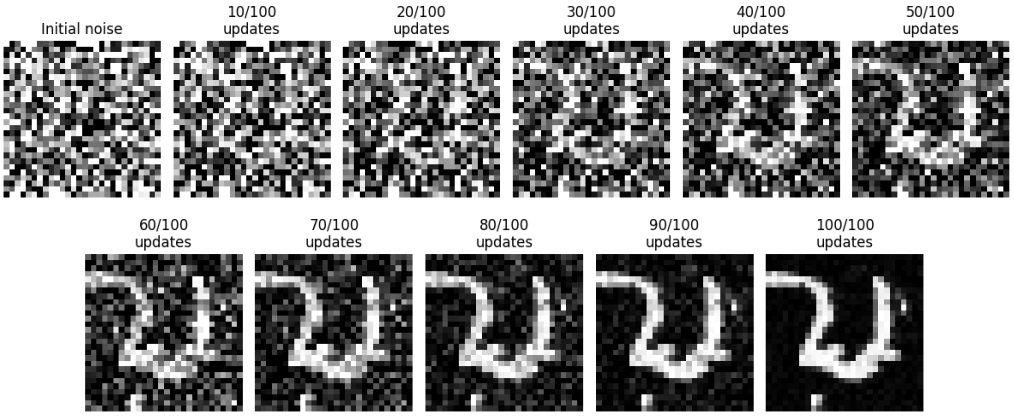


Figure 3. A requested digit 7 develops over a 100-step generation loop. Read the top row first, then the bottom row.

The early panels are dominated by random noise. Curved strokes become visible around 40 to 60 updates, and later panels show a darker background and more defined strokes. However, the final shape does not clearly resemble a 7. The model reduced noise progressively, but did not reliably produce the requested digit.

CLIP Evaluation

CLIP stands for Contrastive Language-Image Pre-training. The notebook compares each image with three descriptions: a handwritten version of the requested number, a clear and well-written version of that number, and a blurry or unclear number. The resulting weights express relative preferences within this prompt set.

Requested digit	Samples	Mean clear-prompt weight	Clear-prompt range
0	4	0.86	0.78 to 0.91
1	4	0.89	0.86 to 0.92
6	10	0.789	0.54 to 0.93

Means for digits 0 and 1 are approximate, based on the rounded weights displayed in the figures.

For the ten digit-6 samples, the clear description received the highest weight every time. These values are not recognition accuracy or measured probabilities that an image contains the correct digit. Because this prompt set does not compare all ten digit identities, it cannot establish classification accuracy.

Limitations and Overall Evaluation

The saved Step 6 grid remains noise-like, while Steps 7 and 8 contain more developed strokes. A consistent rerun using the same trained checkpoint would help investigate this difference. The available examples demonstrate partial success in forming handwriting-like shapes, but reliable

digit identity remains unresolved. Future evaluation should combine visual inspection with an independently validated MNIST digit classifier.

Esther Odaibo — Personal Reflection

My contribution was preparing the assessment report covering diffusion concepts, model architecture, training analysis, CLIP evaluation, and practical applications. Analyzing our notebook results helped me understand that decreasing training loss does not always produce recognizable or correctly labeled images. I also learned that high CLIP scores require careful interpretation and should be supported by visual inspection and additional evaluation. I identified limitations and proposed improvements for the team to consider. In future projects, I would participate earlier in implementation and help compare changes through controlled experiments.

KING OWUSU

Data selection, MNIST and loading, the U-Net, the diffusion process, training, generation, and CLIP evaluation. I wrote the assessment answers, the analysis report, and my learning notes on the model components, working from the notebook's outputs.

What went well? Splitting the work this way let us each focus on one thing. Reading James's outputs closely showed me things the loss number alone did not, such as the schedule ending before pure noise.

What was difficult? The generated digits were not readable, which meant my write-up had to be honest about a weaker result than I expected. It was also hard to separate "the model trained well" from "the model generates well," since those are different things.

What I learned. A low loss does not guarantee good samples. A CLIP "clarity" score is not an accuracy score. Skip connections and conditioning placement matter a lot to what a U-Net can do.

What I would do differently. Test the model's output with a digit classifier earlier, and add the conditioning in every block from the start.

James Delgado — Personal Reflection

My main contribution to this project was working through the diffusion notebook from beginning to end, including dataset preparation, model components, training, image generation, and CLIP evaluation. I also helped troubleshoot errors and added comments to explain what the code was

doing. Completing the full notebook helped me see how the different parts depended on one another.

The most challenging part was connecting unfamiliar Python code and mathematical ideas to the purpose of each step. I used AI extensively for explanations and debugging, and I spent about eight hours working through the notebook cell by cell. I found it helpful to ask for explanations from first principles, including where an idea came from, why it was needed, and how information moved through the model. I still need more practice writing this code independently, but I made progress in understanding the process.

One important lesson was that lower training loss does not guarantee recognizable images. Our model produced visible strokes, but many samples did not resemble the requested digit. I also learned to interpret CLIP results carefully. A high weight for the “clear” description did not mean that the model had generated the correct number. Comparing the images with the reported scores helped me understand the limits of those measurements.

For a future project, I would check generated samples earlier, keep saved outputs consistent with the final notebook run, and establish clearer group responsibilities at the beginning. I would also break the work into smaller experiments so I could connect each change to its effect. My goal is to keep building enough understanding to explain, modify, and eventually implement more of the model independently.